區塊鏈原理與技術

Principles and Technology of Blockchain

平时笔记, *2026* 年春季学期

Student Lewis Xuan, *BUPT.*

What is needed is an electronic payment system based on cryptographic proof instead of trust, allowing any two willing parties to transact directly with each other without the need for a trusted third party.

— Satoshi Nakamoto

Lewis Xuan

密码学基础

Chapter 1

本节主要接收介绍区块链中使用的密码学基础知识，主要包括Hash函数、对称加密与非对称加密、RAS算法以及数字签名与数字证书等内容。事实上并没有详细讨论一些深入、晦涩的密码学理论，主要以科普一些基本的概念为主。

1.1. Hash函数与Hash算法

Hash函数是区块链中最重要的密码学工具之一，它是一种将任意长度的输入数据映射为固定长度输出的函数。常常是将任意长度的输入数据映射为一个64位十六进制的哈希值，并且要求**不同的字节流映射后的值几乎没有冲突**。Hash函数具有以下几个重要的性质：

- 输入数据的微小变化会导致输出哈希值的巨大变化。
- 输出的哈希值的长度是固定的，无论输入数据的长度如何。
- 哈希函数是单向的，即从哈希值无法逆推出原始输入数据。（Hiding，隐秘性）
- 哈希函数具有**抗碰撞性**（Collision Resistance），即很难找到两个不同的输入数据映射为相同的哈希值。换言之，不能通过人为构造输入数据来产生相同的哈希值。
- 给定输入x和目标哈希结果y，希望找到某个随机数z，使得$H(x\|z) = y$，只能通过暴力搜索方法,**谜题友好性**（Puzzle Friendly）。
- 一次哈希运算的速度非常快，时间非常短 （高效性，Difficult to solve, but easy to verify.）

```
hello,world -> 3CB95CFBE1035BCE8C448FCAF80FE7D9

hello world -> 5EB63BBBE01EEED093CB22BB8F5ACDC3
```

Figure 1: Hash函数的性质示例图

从上图可以看出，输入数据的微小变化会导致输出哈希值的巨大变化，这就是Hash函数的敏感性。此外，输出的哈希值的长度是固定的，无论输入数据的长度如何，这也是Hash函数的重要性质之一。最后，哈希函数是单向的，即从哈希值无法逆推出原始输入数据，这也是Hash函数的重要安全性保障之一。

定义 1.1. *Hash碰撞*

Hash碰撞是指两个不同的输入数据映射为相同的哈希值的情况，但实际上这种情况十分少见，只是理论上存在这种可能性，现实中很难找到两个不同的输入数据映射为相同的哈希值。因此，Hash函数在区块链中被广泛应用于数据的完整性验证和数字签名等方面。

常见的 Hash 算法包括 MD5、SHA-1、SHA-256 等，其中 SHA-256 是比特币中使用的 Hash 算法，它具有较高的安全性和较快的计算速度，因此被广泛应用于区块链中。以太坊中采用了 SHA3 哈希函数族的来源——keccak 族的 keccak256 算法。

由于 Hash 算法具有单向性以及抗碰撞性，因此在现实中常常可以用于知识证明或者防篡改等等应用。比如，利用文件的 Hash 值来验证文件的完整性。如果文件的内容发生了任何改变，那么它的 Hash 值也会发生改变。

又由于 Hash 函数具有谜题友好性，因此被用于进行垃圾邮件的筛选，每个用户发送一封邮件，必须解决一个数学问题，找到一个随机数s，使得邮件与这个随机数拼接的哈希值满足条件$< T$，其中$T \in \mathbb{Z}$。因此对于来及邮件的发送者来说，发送大量邮件具有很大的时间成本（**Prove of Work, 工作量证明**），因此可以有效抑制来及邮件的发送。

1.2. 对称加密与非对称加密

在不安全的传输链路上进行消息的传输时，需要对传输的信息进行加密。加密分为对称加密和非对称加密两种方式。对称加密是指加密和解密使用同一个密钥的加密方式，而非对称加密则是指加密和解密使用不同的密钥的加密方式。对称加密时由于秘钥双方一致，因此需要通过安全的方式进行密钥交换，否则可能会被攻击者窃取密钥，从而导致加密信息的泄露。常见的对称加密算法包括 AES、DES 等。

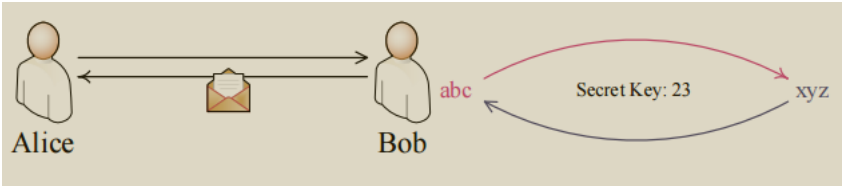

Figure 2: 对称加密示例图

上面的图片使用字母表移位的方式进行加密，比如秘钥是 23，发送的原文为 abc，那么加密后的密文就是 xyz。对于对称加密来说，发送方和接收方需要事先共享一个密钥，这个密钥必须通过安全的方式进行交换，否则可能会被攻击者窃取，从而导致加密信息的泄露。对称加密的保密特性并不是很好，因此又提出了非对称加密的方法，下面介绍一个是一个简单的非对称加密的算法示例。

设若原文为 123456，公钥为 42269，私钥为 11829，加密的算法为与公钥相乘再与 1000000 取模，即$y = (123456 \times 42269) \bmod 1000000$，解密的算法为与私钥相乘再与 1000000 取模，即$x = (y \times 11829) \bmod 1000000$。在这个例子中，公钥和私钥是不同的，因此加密和解密使用不同的密钥，这就是非对称加密的特点。这个例子之所以能够从密文中回复出原文，是因为$42269 \times 11829 \bmod 1000000 = 1$，因此加密和解密的算法是互逆的。

但是这是一个脆弱的加密系统，因为对于攻击者来说，密文和公钥实际上都是公开的，而加密的算法也是可以知道的，因此对于攻击者来说，要想知道原文只需要解下面这个方程即可：

$$y = (x \times 42269) \bmod 1000000$$

其中x为原文，y为密文，上面这个方程使用辗转相除法很容易得到解。

下面介绍一个实际中使用的非对称加密算法——RSA 算法。RSA 算法是由 Ron Rivest、Adi Shamir 和 Leonard Adleman 在 1977 年提出的一种非对称加密算法，它的安全性基于大数分解的困难性。算法的理论基础如下。

1.3. RAS 算法

RSA 算法主要依赖于数论中的一些定理和算法，主要包括中国剩余定理、裴蜀定理、费马小定理等。下面分别介绍这些定理和算法的基本原理以及它们在 RSA 算法中的应用。

1.3.1. 中国剩余定理

在《孙子算经》中有这样一个问题："今有物不知其数，三三数之剩二，五五数之剩三，七七数之剩二，问物几何？用白话描述就是，现在有一个数不知道是多少，只知道这个数除以 3 余 2，除以 5 余 3，除以 7 余 2， 问这个数是多少？

设这个未知数为x，根据题意可以得到以下三个同余方程：

$$x \bmod 3 = 2$$
$$x \bmod 5 = 3$$
$$x \bmod 7 = 2$$

中国剩余定理中的一个重要结论是，如果模数（上面的例子中为$3,5,7$，假设有n个模数为$n_1, n_2, ..., n_n$）之间两两互质，那么这个同余方程组在零到模数的乘积减一范围内有唯一解，在模数乘积到二倍模数乘机范围内又有一解，以此类推。对于上面的例子来说，模数$3,5,7$之间两两互质，因此这个同余方程组在0到$3 \times 5 \times 7 - 1 = 104$范围内有唯一解，在$105$到$209$范围内又有一解，以此类推。

因此可以确定上面孙子算经中的例子未知数在0到104范围内的唯一解为23，在105到209范围内的唯一解为128，以此类推。

中国剩余定理可以进行简单的加密，比如下面有 5 个两两互质的数$3,5,7,11,13$，可以将一个数x加密为一个五元组$(x \bmod 3, x \bmod 5, x \bmod 7, x \bmod 11, x \bmod 13)$，这个五元组就是$x$的加密结果。对于攻击者来说，要想知道原文$x$，只需要解下面这个同余方程组即可：

$$x \bmod 3 = a_1$$
$$x \bmod 5 = a_2$$
$$x \bmod 7 = a_3$$
$$x \bmod 11 = a_4$$
$$x \bmod 13 = a_5$$

其中a_1, a_2, a_3, a_4, a_5为加密结果中的五个元素。由于模数之间两两互质，因此这个同余方程组在0到$3 \times 5 \times 7 \times 11 \times 13 - 1 = 30029$范围内有唯一解，在$30030$到$60059$范围内又有一解，以此类推。因此对于攻击者来说，要想知道原文x，需要解上面这个同余方程组，这个同余方程组的解法的时间复杂度是$O(n^2)$，其中n为模数的个数，因此对于攻击者来说，要想知道原文x，需要花费很长的时间来解上面这个同余方程组，这就是中国剩余定理在 RSA 算法中的应用之一。

或者像是 PPT 中的那样，选取五个两两互质的模数$7,10,11,13,17$，将一个数x加密为五个秘钥为$(7, x \bmod 7), (10, x \bmod 10), (11, x \bmod 11), (13, x \bmod 13), (17, x \bmod 17)$，加密的数$x$选取的范围在$(13 \times 17, 11 \times 7 \times 10)$，攻击者要想拿到数据$x$需要至少获取三组秘钥才行。

1.3.2. 裴蜀定理

裴蜀定理是数论中的一个重要定理，它的内容是：给定两个互质的数a和n，存在一个未知数x使得下列方程组有解：

$$\begin{gathered} a \times x \bmod n = 1 \\ a \times x \bmod n = 2 \\ a \times x \bmod n = 3 \\ \cdots \\ a \times x \bmod n = n-1 \end{gathered}$$

事实上只要第一个方程成立即可满足剩下$n-2$个方程成立。从裴蜀定理的角度重新审视中国剩余定理，原题中的三个同余方程组可以视为下面几个方程组的解，找到一个是5,7 公倍数的数，且模三余二；找一个是 3,7 公倍数的数，且模五余三；找一个是 3,5 公倍数的数，且模七余二。即

$$\begin{gathered} 5 \times 7 \times x \bmod 3 = 2 \\ 3 \times 7 \times x \bmod 5 = 3 \\ 3 \times 5 \times x \bmod 7 = 2 \end{gathered}$$

从上面的三个方程可以看出均满足裴蜀定理的条件，因此这三个方程组都有解。

裴蜀定理的求解方法可以使用扩展的辗转相除法来求解，扩展的辗转相除法的时间复杂度是$O(\log n)$，其中n为模数的大小，因此对于攻击者来说，要想知道原文x，需要花费很长的时间来解上面这个方程组。

例子 1.1. 裴蜀定理的求解示例

设有方程

$$115 \times x \bmod 367 = 1$$

使用扩展的辗转相除法来求解这个方程，首先计算367除以115的商和余数：

$$367 - 3 \times 115 = 22 \tag{1}$$

上式两端再乘 5，得到

$$5 \times 367 - 15 \times 115 = 110 \tag{2}$$

即

$$5 \times 367 - 16 \times 115 = -5 \tag{3}$$

3 式的四倍加上 1 式，得到 $21 \times 367 - 67 \times 115 = 2$ 四式的二倍再加上三式，得到

$$47 \times 367 - 150 \times 115 = -1 \tag{4}$$

因此解得方程$115 \times x \bmod 367 = 1$的解为$x = 150$。

1.3.3. 费马小定理

费马小定理主要有两条性质：

性质 1： 如果n是一个质数，则对于任意的整数a，则$a^n - a$均是n的倍数，即$a^n \bmod n = a \bmod n$。

性质 2： 如果n、a是互质的，对于任意整数i，则$a^i \bmod n$呈现出$n-1$的周期性。

1.3.4. RSA 算法流程

通过上面的三个定理构建起RSA算法的理论基础之后，下面介绍RSA算法的具体流程。RSA 算法主要包括密钥生成、加密和解密三个步骤。

密钥生成：首先选择两个大质数p和q，计算出$n = p \times q$和$\varphi(n) = (p-1) \times (q-1)$。选取要加密的原文$a$。选取一个合适的常数$k$，满足

$$a^{i+k \times \varphi(n)} \bmod n = 1$$

换言之就是选取一个质数e使之满足与$\varphi(n)$互质。同时满足

$$e \times d \bmod \varphi(n) = i$$

求解出d，其中d为私钥，(n, e)为公钥。

实际上就是在对$i + k \times \varphi(n)$在质因数分解，对于攻击者来说，$\varphi(n)$是未知的，而$\varphi(n)$又来自于p和q，因此攻击者要想知道原文a，需要对n进行大数分解，找到p和q，从而计算出$\varphi(n)$，最后在进行质因数分解求解出d，这个过程的时间复杂度是指数级的，因此对于攻击者来说，要想知道原文a，需要花费很长的时间来进行大数分解，这就是RSA算法的安全性保障之一。

目前挑选两个质数计算乘机简单，而对一个很大的质数进行质因数分解目前人没有很好的办法，除了遍历搜索。

1.4. 数字签名与数字证书

上面介绍了对称加密与非对称加密的基本原理以及RSA算法的流程，RSA目前还是安全的，但是仅有对称加密和非对称加密事实上也不能很好的保证通讯的安全。正常安全的通讯应该是Alice发送给Bob他自己的公钥，而Bob也将自己的公钥发送给Alice，双方都使用对方的公钥进行加密通讯，这样就可以保证通讯的安全性了。但是在实际中，攻击者可能会冒充Alice或者Bob来进行攻击，比如攻击者对网络中传输的秘钥进行截取，截取到Alice的公钥后进行保留，将自己假的Bob的公钥发送给Alice，截取到Bob的公钥后进行保留，将自己假的Alice的公钥发送给Bob，自己则保留Alice和Bob假的私钥，以后截取和篡改Alice和Bob的通讯内容。为了防止这种攻击，数字签名和数字证书就被提出了。

数字签名是指发送方使用自己的私钥对消息进行签名，接收方使用发送方的公钥来验证签名的真实性，从而保证消息的完整性和真实性。数字签名起到防篡改的作用，但是首先要正确传输公钥，因此又引入了数字证书，数字证书是由权威机构颁发的，用于证明公钥的真实性和合法性的电子文档。数字证书中包含了公钥、持有者的身份信息以及颁发机构的签名等内容，接收方可以通过验证数字证书来确认公钥的真实性，从而保证通讯的安全性。在发送公钥时，首先应当去颁发数字证书的机构获取数字证书，颁发机构会对发送方的身份进行验证，确认发送方的身份合法后才会颁发数字证书，证明自己是自己，之后使用数字证书中的私钥加密自己的公钥进行放松，目标用户使用正版电脑上存储的对应数字证书机构获取的公钥来进行解密，获得对方的公钥，从而进行安全的通讯。

区块链体系结构

Chapter 2

区块链是一个去中心化、分布式、不可被更改、可追溯、P2P 的一个数位交易网络。

2.1. 区块链的组成

区块链是一个链式的结构，每个区块包含了一个区块头和一个区块体。区块头包含了前一个区块的哈希值、时间戳、随机数等信息，区块体包含了交易数据和其他相关信息。每个区块都通过哈希值与前一个区块连接起来，形成一个链式结构，这就是区块链的基本组成。

一个区块可以视为是某一段时间内发生的交易的集合，区块链则是这些区块按照时间顺序连接起来形成的一个链式结构。每个区块都包含了前一个区块的哈希值，这样就保证了区块链的不可篡改性，因为如果要修改某个区块，就需要修改后续所有区块的哈希值，这在计算上是非常困难的。

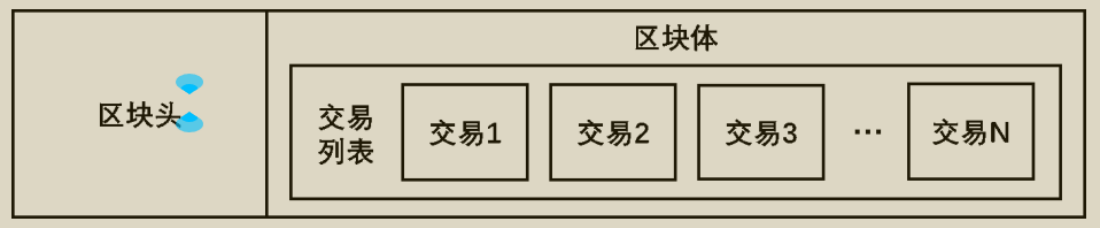

Figure 3: 区块链的组成示例图

首先区块头中存储了这个区块的版本号、前一个区块整体的哈希值、默克尔树根哈希值、时间戳、难度目标和随机数等信息。区块体中存储了这个区块中包含的交易数据和其他相关信息，比如交易的输入和输出、交易的金额等等。

比特币的区块头数据结构		
源码中的命名	大小（字节）	说明
nVersion	4	区块遵守的版本
hashPrevBlock	32	前一区块的哈希值
hashMerkleRoot	32	区块中交易列表的哈希值
nTime	4	产生区块的时间
nBits	4	区块产生的难度目标
nNonce	4	凑齐难度目标的一个随机数

Figure 4: 区块头存储数据组成示例图

区块头中保存有上一个区块整体的哈希值，这样就保证了区块链的不可篡改性，因为如果要修改某个区块，就需要修改后续所有区块的哈希值，这在计算上是非常困难的，同时还要保证数以万计的全量节点均修改成功，这在现实中几乎是不可能的。

hashMarkle 树根哈希值是通过对区块体中所有交易数据进行哈希运算得到的一个哈希值，这样就保证了区块体中交易数据的完整性，因为如果要修改某个交易数据，就需要修改整个区块体的哈希值，这在计算上也是非常困难的。Markle tree 是一种二叉树结构，每个叶子节点存储一个交易数据的哈希值，每个非叶子节点存储其子节点的哈希值，最终根节点存储整个区块体中所有交易数据的哈希值，这样就保证了区块体中交易数据的完整性。具体结构如下图所示：

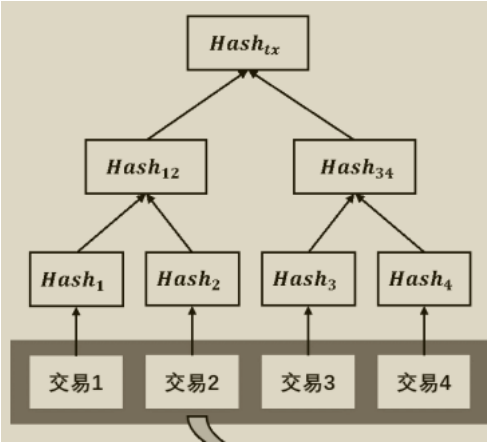

Figure 5: Markle tree 示例图

Markle tree 的优点是可以快速验证某个交易数据是否在区块体中，只需要提供相关的兄弟节点以及祖先节点的 Hash 值即可（Markle Proof），这样就提高了验证的效率。但是要验证一笔交易不在区块中，则需要提供整个区块体中所有交易数据的 Hash 值，这样就降低了验证的效率。

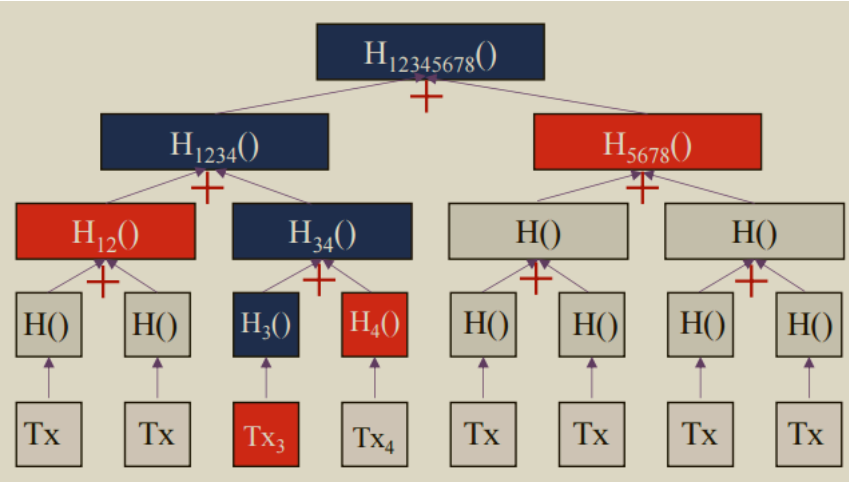

Figure 6: Tx3 交易的 *Markle Proof* 示例图

2.2. 区块链的交易模型

区块链中的交易记录仅仅只记录了交易的输入和输出，并没有记录用户的账户余额等信息，因此区块链中的交易记录并不能直接反映出用户的账户余额等信息。区块链中的交易记录是以一种特殊的方式进行组织和存储的，这种方式被称为 **UTXO（Unspent Transaction Output，未花费交易输出）模型**。由于没有记录用户的账户余额，因此避免双花问题尤为重要。比特币系统中每一笔交易都需要说明交易所用的比特币的来源，即上一笔输出为自己的输入，并且需要提供一个数字签名来证明自己拥有这个输入的所有权，同时还需要说明交易的输出，即将比特币发送给谁（接收方的公钥），以及发送多少比特币。每一笔交易都需要满足输入和输出的平衡，即输入的总金额必须**大于或等于**输出的总金额，否则这笔交易就是无效的。

比特币交易数据结构			
数据项	类型	大小	说明
nVersion	整数	4字节	交易的版本号
vin	变长无符号整型	1-9字节	交易输入数量
	数组	不定	交易的所有输入，按顺序排列
vout	变长无符号整型	1-9字节	交易输出数量
	数组	不定	交易的所有输出，按顺序排列
nLockTime	无符号整型	4字节	用于时间锁定脚本的时间戳

Figure 7: 比特币交易结构示例图

输入的总金额可以通过查询区块链中的交易记录来计算得到，而输出的总金额则是由用户在发起交易时指定的。因此，在比特币系统中，每一笔交易都需要满足输入和输出的平衡，这样就保证了区块链中的交易记录的有效性和可靠性。

为了证明转账后的结果，交易的一个输出包含了两个主要部分——输出的比特币数量和输出的比特币的所有者。如图，前者在比特币中通过一个 64 位的整数来记录，单位是比特币系统中的最小货币单位**聪（Satoshi）**，大小为一亿分之一个比特币。后者我们使用了比特币的**脚本系统**来实现。

比特币交易输出结构			
数据项	类型	大小	描述
nValue	64位无符号整型	8字节	输出中的比特币数量，单位：聪
scriptPubKey	变长无符号整型	1-9字节	解锁输出脚本长度
	字节串	不定	用于锁定的脚本，能够解锁脚本才使用这个输出

Figure 8: 比特币交易输出示例图

交易的输出脚本起到锁定交易的作用，需要交易的目标用户使用输入脚本进行解锁交易，才能完成交易的转账。输入脚本和输出脚本的组合构成了比特币的**脚本系统**，这个系统提供了一种灵活的方式来定义交易的条件和规则，从而实现了比特币系统中的各种复杂的交易类型，比如多重签名交易、时间锁定交易等等。

比特币交易的输入来自于以往过去的交易输出，因此每一笔交易都需要引用之前的交易输出作为输入，这样就形成了一个链式的结构，这也是区块链的基本特征之一。

比特币交易输入结构				
数据项	类型	大小	描述	
prevout	哈希值	32字节	引用上一个交易输出	上一个交易的哈希
	无符号整型	4字节		输出在上一个交易的位置
scriptSig	变长无符号整型	1-9字节	解锁脚本长度	
	字节串	不定	用于解锁上一个输出的脚本	
nSequence	无符号整型	4字节	序列号，在一些解锁的功能中使用	

Figure 9: 比特币交易输入示例图

图中上一笔交易输出的位置含义是第几个转账操作，以此转账可以有多个输入和多个输出，输入和输出的数量没有限制，因此可以实现非常复杂的交易类型，比如多重签名交易、时间锁定交易等等。

下面是一个简单的交易链条指向示例图：

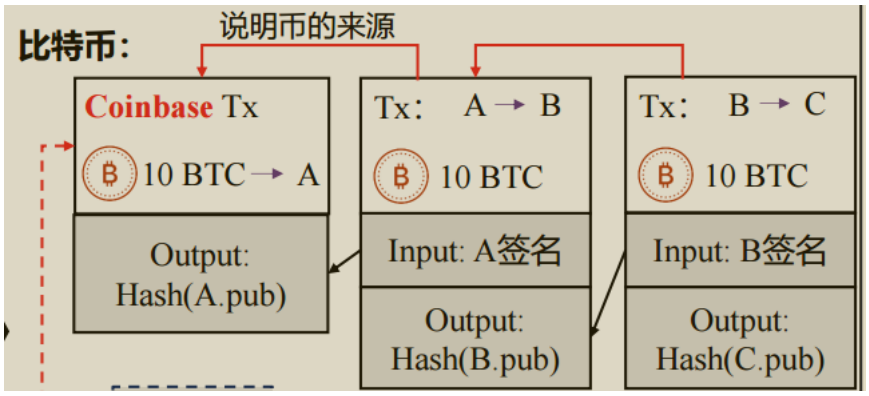

Figure 10: 比特币交易链条示例图

上面图中第一笔交易为 **Coinbase 交易**（铸币交易），即挖矿奖励交易，这笔交易没有输入，只有一个输出，输出比特币的数量为挖矿奖励数量与上一个区块中收取的交易费，输出的比特币的所有者是矿工的公钥。第二笔交易引用了第一笔交易的输出作为输入，第三笔交易引用了第二笔交易的输出作为输入，以此类推，形成了一个链式的结构。

在一个交易记录中，输出为目标账户的公钥，输入为上一笔来源交易的输出，输入脚本为解锁脚本，输出脚本为锁定脚本。

为了保证转账发起者的输入是可靠的合法的，则要使用到 **UTXO 模型**，这个模型记录了所有现存比特币的去向，即每一笔交易的**未花费输出**都被记录下来，如果这个输出没有被后续的交易引用作为输入，那么这个输出就是一个未花费的交易输出（UTXO），只有这些 UTXO 才是合法的输入，才能被用来发起新的交易。

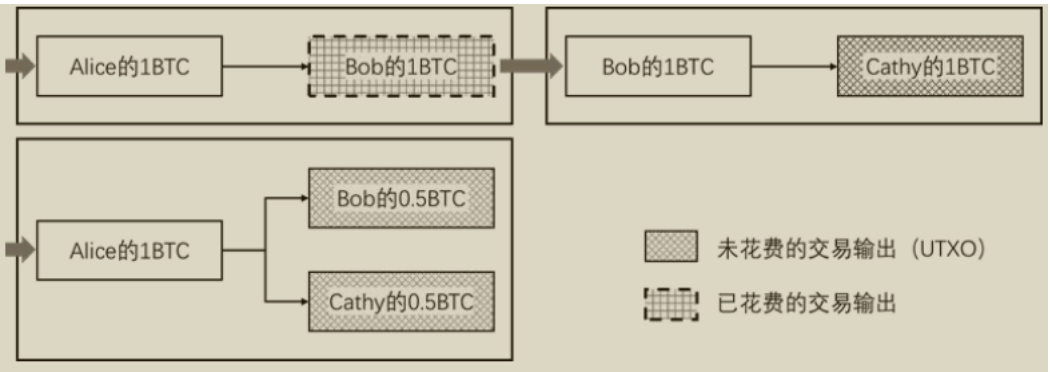

Figure 11: UTXO 模型示例图

2.3. 区块链交易脚本的种类与基本原理

区块链中的交易脚本是用来检验交易合法性，保证交易可靠的一种工具。交易脚本主要分为输入脚本和输出脚本两种类型，输入脚本用来解锁交易，输出脚本用来锁定交易。交易脚本本质是一种基于栈的执行语言。输入脚本先入栈，随后输出脚本入栈，以此从栈顶开始执行，脚本不报错，就说明交易合法。下面详细叙述各种脚本的应用。

2.3.1. P2PK（Pay to Public Key）脚本

P2PK 脚本是最简单的一种交易脚本，它的输出脚本直接包含了接收方的公钥，输入脚本则包含了发送方的数字签名。P2PK 脚本的输出脚本格式如下：

- 输入脚本：PUSHDATA SIG 压入转账目标用户的数字签名
- 输出脚本：
  PUSHDATA PublicKey 压入转账目标用户的公钥
  CHECKSIG 验证数字签名的有效性

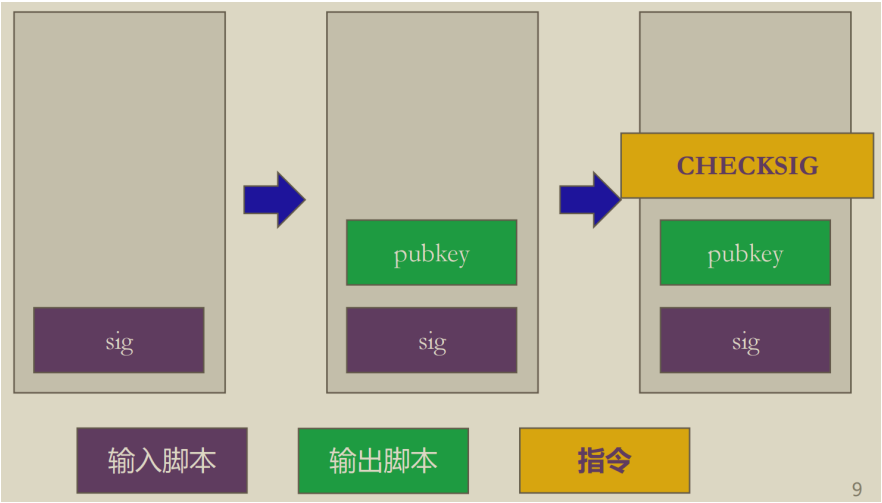

Figure 12: P2PK 脚本示例图

2.3.2. P2PKH（Pay to Public Key Hash）脚本

上面描述的脚本有一个缺点，就是直接使用目标账户的公钥进行传输，为了用户的隐私与保密性，P2PKH 脚本被提出了。P2PKH 脚本的输出脚本格式如下：

- 输入脚本：PUSHDATA SIG 压入转账目标用户的数字签名
- 输出脚本：
  DUP 复制栈顶元素
  HASH160 对栈顶元素进行 Hash 运算
  PUSHDATA PublicKeyHash 压入转账目标用户的公钥哈希值
  EQUALVERIFY 验证栈顶两个元素是否相等
  CHECKSIG 验证数字签名的有效性

该脚本中由输出脚本提供目标账户的公钥 Hash 值，而不是由目标账户直接提供自己的公钥，这样就提高了用户的隐私与保密性，因为攻击者无法通过交易记录直接获取目标账户的公钥，从而无法进行针对性的攻击。同时，输入脚本中仍然包含了发送方的数字签名，这样就保证了交易的合法性和可靠性。

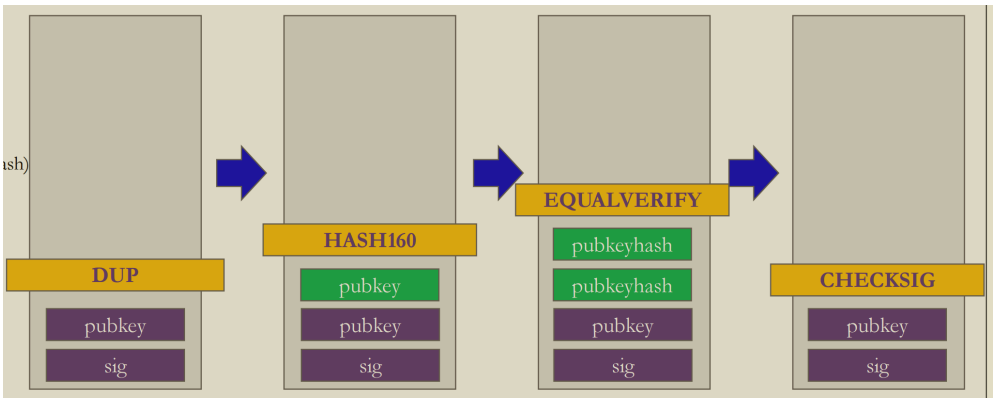

Figure 13: P2PKH 脚本示例图

上图中的最后一步是为了验证给出输入脚本的是拥有该秘钥的“本人”。

2.3.3. MultiSig 脚本（P2SM 脚本）

N个公钥，M（$M \leq N$）个验证通过即可

- **输入脚本**：压入多个账户的数字签名。但是第一行不能写有效命令，从第二行开始写压入命令，**这是一个 BUG**
- **输出脚本**：压入所有N个用户的公钥，进行验证。

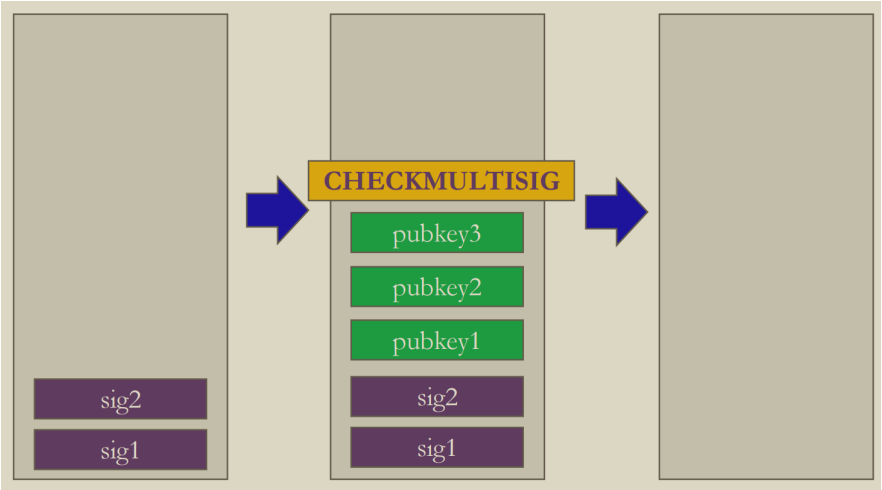

Figure 14: P2SM 脚本示例图

2.3.4. P2SH（Pay to Script Hash）脚本

P2SH 脚本是一种更为复杂的交易脚本，它的输出脚本包含了一个脚本哈希值，而不是直接包含目标账户的公钥或者公钥哈希值。P2SH 脚本具体分为下面三个脚本。

- **赎回脚本（redeemed script）**：由 P2PK、P2PKH 或 MultiSig 脚本（多重签名）组成，需要**两阶段验证**。
- **输入脚本**：压入目标账户的数字签名与赎回脚本。
- **输出脚本**：先对栈顶的输出脚本进行 Hash 操作，再压入改脚本的 Hash 值（一般由转账的目标用户提供），随后将两个 Hash 值进行比较，**完成第一步认证**。

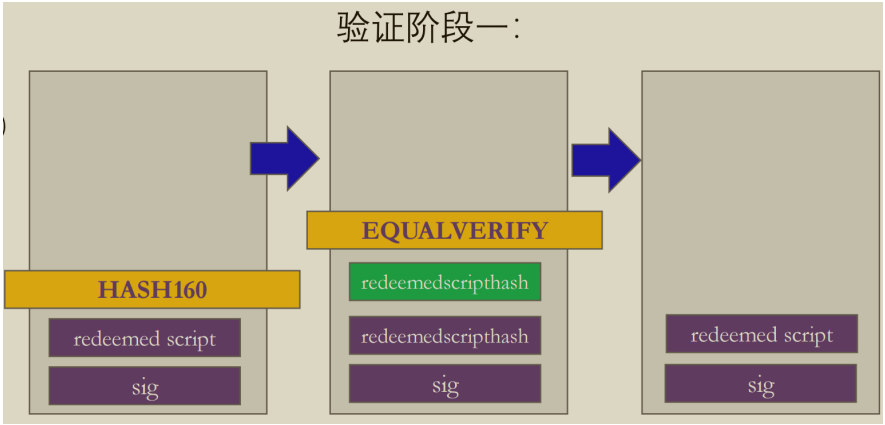

Figure 15: P2SH 验证阶段一

验证阶段二中，对赎回脚本进行操作，对不同类型的赎回脚本，有不同的验证方式。

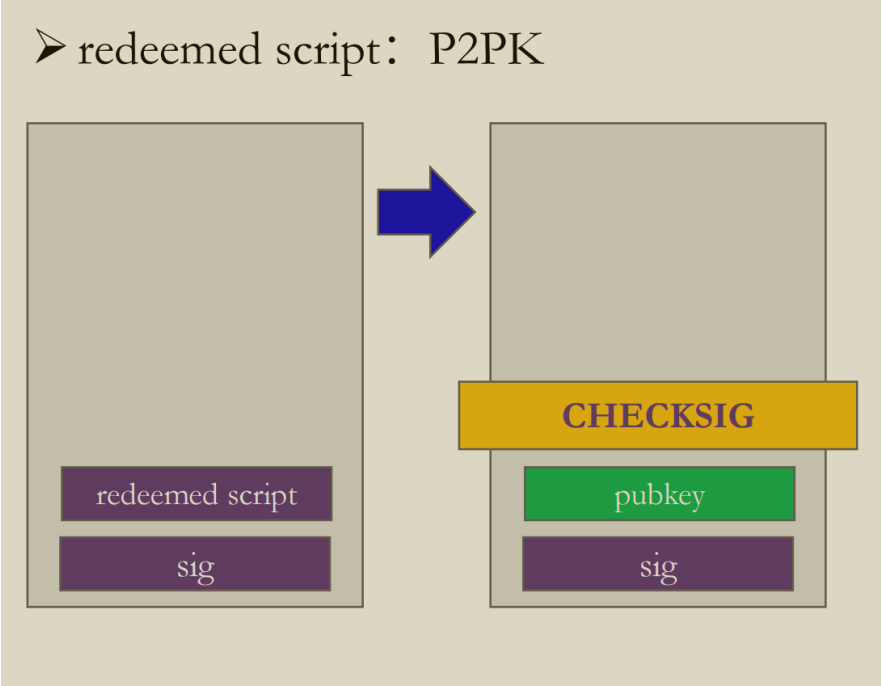

Figure 16: 赎回脚本为 *P2PK* 时的验证阶段二

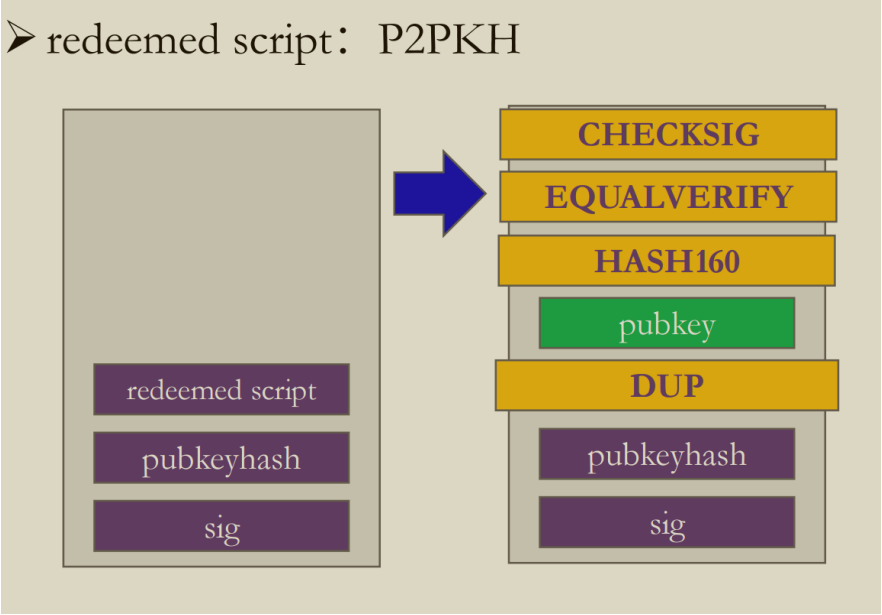

Figure 17: 赎回脚本为 *P2PKH* 时的验证阶段二

2.3.5. NULLDATA 脚本

输出脚本中加入OP_RETURN语句，则在任何情况下均终止交易，比如在一个交易中通过在输出脚本中写OP_RETURN语句并携带一定的信息，就可以将这个信息永久的保留在区块链中，一般来说这种操作强制花费0 BTC，但是如果强制往里面塞钱，会导致比特币燃烧，作废永远无法使用（**Proof of Brun**）

2.4. 区块链的一致性问题

比特币系统的节点分为全量节点与轻量节点，并且任何一个地址都可发起交易，交易将广播给全量节点，等待打包上链，因此需要保证全网所有全量节点中记录的区块链均一致，所有的账本均统一。由此引出下面两个统一机制

2.4.1. 投票选举

由全网所有全量节点保存各自最新的区块链于本地，各自校验交易的完整性与合法性，在将自己的新的区块发送给其他的全量节点，进行选举，票数过半者成为下一个新的区块。

但是这样有几个问题，最关键的是可能存在**女巫攻击**，即一个人注册多个账号（比特币的账号注册是简单的，只要有一个对应的公钥、私钥即可），进行投票，干扰投票的公平性。

2.4.2. Proof of Work

为了保证新的区块产生的合理性与公平性，比特币系统采用工作量证明（**Proof of Work**）来选举残生下一个新的区块。这里就需要使用到上面区块头中所存储的最后两个字段nBits(区块产生难度)与nNonce（凑齐难度目标的一个随机数），为了产生一个新的区块，需要在已经确定的父区块的基础上进行挖掘，也即要满足条件，调整随机数，使得区块头的Hash值小于给定的目标难度，同时还可以改动Coinbase交易中所携带的Message，来改变Markle根Hash值，进而改变父区块头部Hash值来满足挖矿要求。这种满足条件的搜索只能进行遍历搜寻，属于Hash运算的**谜题友好性（Puzzle Friendly）**。